Three-Dimensional Choreographic Orbits: Computational Specification

CompSpec — v0.2 — Companion to RDD v0.2 and MathFrame v0.1

e5bed97b3349 • March 2026

Contents

1 Software Stack and Hardware Target **1**
- 1.1 Primary framework: JAX . . . 1
- 1.2 Supporting libraries . . . 1
- 1.3 Hardware target . . . 2

2 Mathematical Encoding **2**
- 2.1 Fourier representation of the curve . . . 2
- 2.2 Symmetry constraint encoding . . . 2
  - 2.2.1 Worked example: C_{3v} with $N = 3$ . . . 2
  - 2.2.2 Implementation: mode-selection projector . . . 3
- 2.3 Action functional in JAX . . . 5

3 Phase 1: Action Minimisation **6**
- 3.1 Initialisation strategy . . . 6
- 3.2 L-BFGS via Optax . . . 7
- 3.3 Candidate filtering . . . 7

4 Phase 2: Orbit Refinement **7**
- 4.1 Yoshida 6th-order symplectic integrator . . . 7
- 4.2 Newton–Raphson periodicity correction . . . 8
- 4.3 Symmetry verification . . . 8

5 Phase 3: Knot Classification Pipeline **8**
- 5.1 Spacetime trace construction . . . 8
- 5.2 Projection and crossing diagram . . . 8
- 5.3 Polynomial invariants via SnapPy . . . 8

6 Phase 4: Stability Analysis **9**
- 6.1 Variational equations . . . 9
- 6.2 Floquet classification . . . 9
- 6.3 Lyapunov exponents . . . 10

7 Phase 5: Visualisation Pipeline **10**
7.1 Tube rendering 10
7.2 Video export 10
7.3 Static publication figures 10

8 Catalogue Format **10**

9 Development Milestones **11**

---

1 Software Stack and Hardware Target

1.1 Primary framework: JAX

Design Decision 1. All action evaluation and gradient computation will be implemented in **JAX** (`jax>=0.4.25`), rather than NumPy, PyTorch, or SciPy.

JAX is the correct choice for three compounding reasons. First, the action functional $\mathcal{A}[\gamma]$ is a smooth function of the Fourier coefficients; JAX's `jax.grad` and `jax.hessian` provide exact gradients and Hessians at machine precision without finite-differencing. Second, `jax.vmap` vectorises the action evaluation over an entire batch of candidate coefficient vectors with zero loop overhead, enabling thousands of random initialisations to be evaluated simultaneously. Third, `jax.jit` compiles the entire pipeline — Fourier synthesis, pairwise distance computation, action summation — to XLA and dispatches it to the GPU without data movement overhead.

1.2 Supporting libraries

Library	Version	Role
`jax[cuda12]`	$\geq 0.4.25$	Core autodiff, JIT, GPU dispatch
`optax`	$\geq 0.2.2$	L-BFGS and Adam optimisers
`scipy`	≥ 1.13	Newton–Raphson, eigenvalues
`snappy`	≥ 3.1	Knot invariants (Alexander, HOMFLY-PT)
`pyvista`	≥ 0.44	3-D tube renders, symmetry-axis overlays
`ffmpeg-python`	≥ 0.2	4K/60fps video export
`numpy`	≥ 1.26	Data I/O, catalogue serialisation
`h5py`	≥ 3.11	HDF5 storage for orbit catalogue

1.3 Hardware target

The primary compute target is a single workstation equipped with an NVIDIA RTX 5090 (Blackwell, 32 GB VRAM) and 192 GB system RAM. Key implications:

- The Fourier truncation limit K can be pushed to $K = 256$ or beyond without memory pressure, capturing highly irregular curve geometries that lower-K truncations would miss.

- With `vmap`, a batch of $B = 8192$ random initialisations can be evaluated in a single forward pass; the GPU's 3352 tensor cores parallelise all B action evaluations concurrently.
- The 192 GB system RAM allows the full symmetry-constraint manifolds for larger N to be pre-enumerated and held in RAM during search.

2 Mathematical Encoding

2.1 Fourier representation of the curve

The single curve $\gamma : [0, T] \to \mathbb{R}^3$ is represented by a truncated real Fourier series:

$$\gamma(t) = \mathbf{a}_0 + \sum_{k=1}^{K} \left[\mathbf{a}_k \cos\left(\tfrac{2\pi kt}{T}\right) + \mathbf{b}_k \sin\left(\tfrac{2\pi kt}{T}\right) \right], \qquad \mathbf{a}_k, \mathbf{b}_k \in \mathbb{R}^3.$$

The parameter vector is $\theta = (\mathbf{a}_0, \mathbf{a}_1, \mathbf{b}_1, \ldots, \mathbf{a}_K, \mathbf{b}_K) \in \mathbb{R}^{3(2K+1)}$. The centre-of-mass condition $\sum_i \mathbf{q}_i = 0$ for all t enforces $\mathbf{a}_0 = \mathbf{0}$, reducing $\theta \in \mathbb{R}^{6K}$. The time discretisation uses $M = 8K$ uniformly spaced quadrature points; the action integral is approximated by the rectangle rule, whose error is $O(K^{-p})$ for a curve of Sobolev regularity p.

Design Decision 2. Default truncation: $K = 64$ for Phase 1 searches. Raise to $K = 128$ for refinement and $K = 256$ for near-collision orbits flagged during Phase 1. Monitor convergence via $\|\mathbf{a}_K\|, \|\mathbf{b}_K\| < 10^{-8}$.

2.2 Symmetry constraint encoding

Let $G \subset O(3) \times \mathbb{Z}/N\mathbb{Z}$ be the target symmetry group. Each generator $(R, \sigma) \in G$ acts on θ by:

$$(R, \sigma) \cdot \gamma(t) = R\,\gamma\left(t - \sigma\,\tfrac{T}{N}\right).$$

In Fourier space this becomes a *linear* constraint on θ. Concretely, applying R to the coefficient vectors and shifting t by $\sigma T/N$ multiplies mode k by the phase factor $e^{2\pi i k\sigma/N}$. The fixed-point subspace $\mathrm{Fix}(G)$ is the intersection of the null spaces of $(I - \rho(g))$ for each generator $g \in G$, where ρ is the induced representation on $\mathbb{R}^{6K}$.

2.2.1 Worked example: C_{3v} with $N = 3$

The group C_{3v} is generated by a 120° rotation R_3 about the z-axis and a reflection σ_v through the xz-plane. With $N = 3$ and phase shift $\sigma = 1$:

1. $R_3 \cdot \gamma(t) = R_3\,\gamma(t - T/3)$: in the helicity basis $z_\pm = x \pm iy$, mode k acquires phase $e^{2\pi i(1-k)/3}$ in the xy-plane and $e^{-2\pi ik/3}$ on the z-axis. Survival requires these phases to equal 1, giving $k \equiv 1 \pmod 3$ for xy-components and $k \equiv 0 \pmod 3$ for z-components (MathFrame Proposition 4.1).
2. $\sigma_v \cdot \gamma(t) = \sigma_v \gamma(t)$: reflects $y \to -y$, forcing $(\mathbf{a}_k)_y = 0$ and $(\mathbf{b}_k)_x = 0$ for all surviving modes.

The net effect is that $\dim \mathrm{Fix}(C_{3v}) = 4K/3 + O(1)$, a reduction by factor $\approx 2/3$ relative to the unconstrained $6K$-dimensional space. Icosahedral constraints with $N = 12$ reduce the space by factor $\approx 11/12$, leaving $O(K)$ free parameters at fixed order.

2.2.2 Implementation: mode-selection projector

Rather than assembling the Reynolds operator $P_G = |G|^{-1} \sum_{g \in G} \rho(g)$ at cost $O(|G|\, K^2)$, we exploit the explicit mode-selection rules derived in MathFrame [5] Propositions 4.1–4.3. For every point group G the fixed-point subspace $\mathrm{Fix}(G)$ is spanned by a subset of Fourier modes that can be identified by a single modular-arithmetic lookup per mode index. The resulting `build_projector` routine runs in $O(K)$ and returns a boolean mask rather than a dense matrix.

Listing 1: Mode-selection projector (MathFrame §4).

```python
import jax.numpy as jnp
import numpy as np

# Mode-selection rules derived from MathFrame Propositions 4.1-4.3.
# Each entry: (xy_residue, z_residue, modulus)
# Mode k survives in xy iff k % modulus == xy_residue  (None = never)
# Mode k survives in z  iff k % modulus == z_residue   (None = never)
_MODE_RULES = {
    # Cyclic / pyramidal: 3D orbits possible
    "Cn":  lambda n: (1 % n, 0, n),
    "Cnv": lambda n: (1 % n, 0, n),
    # Dihedral / prismatic: z forced to zero (no 3D)
    "Dn":  lambda n: (1 % n, None, n),
    "Dnh": lambda n: (1 % n, None, n),
    # Cubic: sparse mode structure, 3D possible
    "T":   lambda _: (1, 0, 3),
    "Td":  lambda _: (1, 0, 3),
    "O":   lambda _: (1, 0, 4),
    "Oh":  lambda _: (1, 0, 4),
    "I":   lambda _: (1, 0, 5),
    "Ih":  lambda _: (1, 0, 5),
}

def build_projector(group: str, n: int, K: int):
    """
    Return boolean masks (xy_mask, z_mask) of shape (K,) indicating which
    Fourier modes k=1..K survive the symmetry constraint for the given group.

    Parameters
```

```python
    ----------
    group : str   Schoenflies symbol, e.g. 'Cnv', 'T', 'Oh'
    n     : int   Fold number (ignored for cubic groups)
    K     : int   Truncation order

    Returns
    -------
    xy_mask : np.ndarray[bool], shape (K,)
    z_mask  : np.ndarray[bool], shape (K,)
    """
    rule = _MODE_RULES[group](n)
    xy_res, z_res, mod = rule
    ks = np.arange(1, K + 1)
    xy_mask = (ks % mod == xy_res) if xy_res is not None \
              else np.zeros(K, dtype=bool)
    z_mask  = (ks % mod == z_res)  if z_res  is not None \
              else np.zeros(K, dtype=bool)
    return xy_mask, z_mask

def pack_free(a_coeffs, b_coeffs, xy_mask, z_mask):
    """Extract the free (unconstrained) Fourier coefficients into a 1-D vector."""
    xy_a = a_coeffs[xy_mask, :2].ravel()   # x,y components of cos modes
    xy_b = b_coeffs[xy_mask, :2].ravel()   # x,y components of sin modes
    z_a  = a_coeffs[z_mask,  2]            # z component of cos modes
    z_b  = b_coeffs[z_mask,  2]            # z component of sin modes
    return np.concatenate([xy_a, xy_b, z_a, z_b])

def unpack_free(theta_free, K, xy_mask, z_mask):
    """Reconstruct full (K,2,3) coefficient array from free parameters."""
    n_xy = int(xy_mask.sum())
    n_z  = int(z_mask.sum())
    xy_a = theta_free[:2*n_xy].reshape(n_xy, 2)
    xy_b = theta_free[2*n_xy:4*n_xy].reshape(n_xy, 2)
    z_a  = theta_free[4*n_xy:4*n_xy+n_z]
    z_b  = theta_free[4*n_xy+n_z:]
    a = np.zeros((K, 3))
    b = np.zeros((K, 3))
    a[xy_mask, :2] = xy_a
    b[xy_mask, :2] = xy_b
    a[z_mask,   2] = z_a
    b[z_mask,   2] = z_b
    return jnp.array(np.stack([a, b], axis=1))   # (K, 2,
```

```
        3)
```

The optimiser operates entirely on the *free* parameter vector θ_{free} whose dimension is $\dim \mathrm{Fix}(G)$ (typically $O(K)$ with a group-dependent prefactor, as tabulated in §2.2.1). Because the parametrisation is intrinsic to $\mathrm{Fix}(G)$, no projection step is required after gradient updates: every point in the free-parameter space corresponds to a valid symmetry-constrained orbit.

2.3 Action functional in JAX

Listing 2: Core action functional (JAX).

```python
import jax
import jax.numpy as jnp

def build_action(N: int, K: int, M: int, G: float = 1.0, m: float = 1.0):
    """
    Returns a JAX-jittable function action(theta) -> scalar.
    theta: shape (6*K,) -- packed Fourier coefficients [a1,b1,...,aK,bK],
           each a_k, b_k in R^3, centre-of-mass zeroed.
    """
    t = jnp.linspace(0, 1, M, endpoint=False)          # normalised time
    tau = jnp.arange(1, N) / N                          # phase offsets

    @jax.jit
    def action(theta):
        # Unpack coefficients: shape (K, 2, 3)
        coeff = theta.reshape(K, 2, 3)
        ks = jnp.arange(1, K + 1)[:, None]              # (K,1)

        # Synthesise gamma(t): shape (M, 3)
        cos_t = jnp.cos(2 * jnp.pi * ks * t[None, :])  # (K, M)
        sin_t = jnp.sin(2 * jnp.pi * ks * t[None, :])  # (K, M)
        gamma = jnp.einsum('ki,kc->ic', cos_t, coeff[:, 0, :]) \
              + jnp.einsum('ki,kc->ic', sin_t, coeff[:, 1, :])  # (M,3)

        # Kinetic term: (1/2) m |gamma_dot|^2
        gamma_dot = jnp.einsum('k,ki,kc->ic',
                               2*jnp.pi*ks[:, 0], -sin_t, coeff[:, 0, :]) \
```

```
                    + jnp.einsum('k,ki,kc->ic',
                                 2*jnp.pi*ks[:, 0],  cos_t,
                                    coeff[:, 1, :])
        kinetic = 0.5 * m * jnp.mean(jnp.sum(gamma_dot**2,
            axis=-1))

        # Potential term: sum over phase offsets
        def potential_shift(s):
            cos_s = jnp.cos(2 * jnp.pi * ks * s)   # (K,)
            sin_s = jnp.sin(2 * jnp.pi * ks * s)
            gamma_s = jnp.einsum('k,ki,kc->ic', cos_s,
                cos_t, coeff[:,0,:]) \
                      + jnp.einsum('k,ki,kc->ic', sin_s,
                         cos_t, coeff[:,0,:]) \
                      + jnp.einsum('k,ki,kc->ic', cos_s,
                         sin_t, coeff[:,1,:]) \
                      + jnp.einsum('k,ki,kc->ic', sin_s,
                         sin_t, coeff[:,1,:])
            diff = gamma - gamma_s                      #
                (M, 3)
            r = jnp.sqrt(jnp.sum(diff**2, axis=-1) + 1e
                -14)
            return -G * m**2 * jnp.mean(1.0 / r)

        potential = jax.vmap(potential_shift)(tau).sum()
        return kinetic + potential / N

    return action
```

Note on collision regularisation. The softening term 10^{-14} in the denominator prevents gradient explosions during early optimisation but must be removed before final Newton–Raphson refinement. Orbits for which $\min_t |\gamma(t)-\gamma(t+kT/N)| < 10^{-3}$ at convergence are flagged as near-collision and discarded.

3 Phase 1: Action Minimisation

3.1 Initialisation strategy

Algorithm: Batched symmetry-constrained initialisation.

(In) Point group G, body count N, batch size B, scale σ

(1) Precompute Reynolds projector $P_G \in \mathbb{R}^{6K\times 6K}$

(2) Sample $Z \sim \mathcal{N}(0, \sigma^2 I_{B\times 6K})$

(3) $\Theta \leftarrow Z \cdot P_G^\top$ *(project onto* $\mathrm{Fix}(G)$*)*

(4) Normalise each row: $\Theta_i \leftarrow \sigma\Theta_i/\|\Theta_i\|$

(Out) Batch $\Theta \in \mathbb{R}^{B\times 6K}$, all rows in $\mathrm{Fix}(G)$

Default: $B = 8192$, $\sigma = 0.3$ (in units where $G = m = T = 1$). The full batch is evaluated in a single `vmap`ped forward pass.

3.2 L-BFGS via Optax

Gradient descent uses the L-BFGS implementation in `optax` with history length $h = 20$. The gradient $\nabla_\theta \mathcal{A}$ is provided exactly by `jax.grad`. After each step, the update is projected back to $\mathrm{Fix}(G)$:

$$\theta \leftarrow P_G\Big(\theta - \alpha \cdot \text{L-BFGS}(\nabla_\theta \mathcal{A})\Big).$$

Convergence is declared when $\|\nabla_\theta \mathcal{A}\|_2 < 10^{-9}$.

3.3 Candidate filtering

After optimisation, candidates are filtered by:

1. **Action threshold**: retain only candidates with $\mathcal{A} < \mathcal{A}_{\text{planar}}(N)$, i.e. strictly below the best known planar choreography at the same N.
2. **Planarity test**: compute the ratio of the smallest to largest principal moment of the point cloud $\{\gamma(t_i)\}$. Discard if ratio $< 10^{-2}$.
3. **Near-collision filter**: discard if any pairwise separation $< 10^{-3}$ along the refined orbit.
4. **Duplicate detection**: cluster retained candidates by action value (tolerance 10^{-6}) and symmetry group to remove re-discovered copies.

4 Phase 2: Orbit Refinement

4.1 Yoshida 6th-order symplectic integrator

Fourier-space minimisation yields an approximate periodic orbit. It is refined to a true solution of the N-body ODE using a Yoshida 6th-order symplectic integrator [1], which preserves the symplectic structure and gives energy conservation error $O(\Delta t^6)$.

The Yoshida coefficients $(w_0, w_1, \ldots, w_7)$ are fixed by the root of a specific polynomial; we use the Yoshida (1990) set 1 values, stored as compile-time constants. Step size $\Delta t = T/(200K)$.

4.2 Newton–Raphson periodicity correction

Let $\Phi_T : \mathbb{R}^{6N} \to \mathbb{R}^{6N}$ denote the time-T flow map computed by the symplectic integrator. We seek $\mathbf{x}^* \in \mathbb{R}^{6N}$ such that $\Phi_T(\mathbf{x}^*) = \mathbf{x}^*$. Starting from the Fourier candidate, Newton–Raphson iterations:

$$\mathbf{x}^{(n+1)} = \mathbf{x}^{(n)} - \Big[D\Phi_T(\mathbf{x}^{(n)}) - I\Big]^{-1} \Big[\Phi_T(\mathbf{x}^{(n)}) - \mathbf{x}^{(n)}\Big]$$

are iterated until $\|\Phi_T(\mathbf{x}^*) - \mathbf{x}^*\|_\infty < 10^{-12}$. The Jacobian $D\Phi_T$ is computed by integrating the variational equations alongside the orbit (Section 6). Typically 3–5 Newton steps suffice from a good Fourier initial point.

4.3 Symmetry verification

After refinement, the symmetry of the orbit is re-verified numerically: for each generator $(R, \sigma) \in G$, compute $\|R\,\mathbf{q}_1(t) - \mathbf{q}_1(t - \sigma T/N)\|_\infty$ and assert $< 10^{-9}$. Orbits failing this check are re-classified under their actual (smaller) symmetry group.

5 Phase 3: Knot Classification Pipeline

5.1 Spacetime trace construction

The spacetime trace Γ is the closed curve in $\mathbb{R}^3$ obtained by "unwinding" the time axis:

$$\Gamma(\phi) = \Big(\gamma(T\phi/2\pi),\ R_z(\phi)\,\hat{e}_z\Big) \in \mathbb{R}^3, \quad \phi \in [0, 2\pi).$$

In practice we embed Γ in $S^3 \subset \mathbb{R}^4$ using the standard Hopf-type map and project to $\mathbb{R}^3$ via stereographic projection before passing to SnapPy.

5.2 Projection and crossing diagram

From the sampled curve Γ (with $L = 2048$ sample points), we:

1. Compute the writhe $w = \frac{1}{4\pi} \iint \frac{(\Gamma(s)-\Gamma(t))}{|\Gamma(s)-\Gamma(t)|^3} \cdot (d\Gamma(s) \times d\Gamma(t))$ numerically (discretised Gauss integral).
2. Select a projection direction minimising the number of crossings (sampled over 200 random directions on S^2).
3. Compute the Gauss code from the ordered crossing sequence and convert to Dowker–Thistlethwaite (DT) notation.

5.3 Polynomial invariants via SnapPy

Listing 3: Knot invariant computation.

```python
import snappy

def compute_knot_invariants(dt_notation: str) -> dict:
    """
    Given DT notation of the spacetime trace, compute invariants.
    """
    L = snappy.Link(dt_notation)
    K = L.components[0] if L.is_knot() else None

    results = {
```

```
        'alexander': str(L.alexander_polynomial()),
        'homfly_pt': str(L.homfly_polynomial()),
        'determinant': L.determinant(),
        'signature': L.signature(),
    }
    if K is not None:
        M = K.complement()
        results['hyperbolic_volume'] = M.volume() if M.solution_type() == \
            'all_positively_oriented_tetrahedra' else None
    return results
```

6 Phase 4: Stability Analysis

6.1 Variational equations

The linearised equations of motion for a displacement $\delta\mathbf{x}(t)$ from the reference orbit $\mathbf{x}^*(t)$ are:

$$\delta\dot{\mathbf{x}} = J(t)\,\delta\mathbf{x}, \qquad J_{ij}(t) = \left.\frac{\partial^2 H}{\partial x_i \partial x_j}\right|_{\mathbf{x}^*(t)},$$

where H is the N-body Hamiltonian. These are integrated alongside the orbit as a $6N \times 6N$ matrix ODE, yielding the fundamental matrix $\Phi(t)$. The monodromy matrix is $M = \Phi(T)$.

Design Decision 3. The Jacobian $J(t)$ is computed via `jax.hessian` of the Hamiltonian at each timestep rather than by finite differencing, ensuring exact second derivatives.

6.2 Floquet classification

Eigenvalues of M are classified as:

Type	Condition	Interpretation
Elliptic	$\lvert\lambda\rvert = 1$, $\lambda \notin \mathbb{R}$	Linearly stable mode
Hyperbolic	$\lambda \in \mathbb{R}$, $\lvert\lambda\rvert \neq 1$	Unstable mode
Complex-hyp.	$\lvert\lambda\rvert \neq 1$, $\lambda \notin \mathbb{R}$	Unstable, spiralling
Trivial	$\lambda = 1$ (multiplicity ≥ 4)	Symmetry-mandated

An orbit is declared *linearly stable* if all non-trivial eigenvalues are elliptic. The four trivial unit eigenvalues arise from time-translation and spatial-translation symmetry.

6.3 Lyapunov exponents

For linearly unstable orbits, the leading Lyapunov exponent λ_1 is estimated by the standard QR-renormalisation method over 10^4 periods.

7 Phase 5: Visualisation Pipeline

7.1 Tube rendering

Orbital traces are rendered as smooth tubes in $\mathbb{R}^3$ using PyVista:

- **Tube radius**: $0.02\times$ characteristic orbit scale.
- **Colour map**: the tube is coloured by arclength parameter to make the body's progression visually legible.
- **Symmetry axes**: rotation axes are drawn as semi-transparent cylinders; mirror planes as translucent discs.
- **Knot diagram inset**: a 2-D knot diagram of the spacetime trace is composited as an inset in the lower-right corner of each render.

7.2 Video export

Design Decision 4. Output format: `H.264`, 3840×2160 (4K), 60 fps, CRF 18 (visually lossless). Container: `.mp4`.

The animation shows the bodies moving along the pre-rendered tube, with a camera orbit completing one revolution per two choreographic periods. The `ffmpeg-python` wrapper handles frame assembly; PyVista's off-screen renderer (`pyvista.start_xvfb()`) is used in the headless VM environment.

7.3 Static publication figures

Each orbit additionally produces a static 300 dpi PDF figure (via PyVista's `save_graphic`) suitable for journal submission: three orthographic views (top, front, side) plus the 3/4 perspective view, assembled on a 2×2 grid.

8 Catalogue Format

All discovered orbits are stored in a single HDF5 file `catalogue.h5` with the following schema:

Dataset	Shape	Content
`orbits/{id}/q`	$(M, N, 3)$	Positions at M timesteps
`orbits/{id}/p`	$(M, N, 3)$	Momenta
`orbits/{id}/action`	scalar	Action value
`orbits/{id}/period`	scalar	Period T
`orbits/{id}/symgroup`	string	Schönflies label
`orbits/{id}/knot`	string	Knot name or DT notation
`orbits/{id}/alexander`	string	Alexander polynomial
`orbits/{id}/homfly`	string	HOMFLY-PT polynomial
`orbits/{id}/floquet`	$(6N,)$	Monodromy eigenvalues
`orbits/{id}/stable`	bool	Linear stability flag

Initial conditions are stored to full IEEE 754 double precision (64-bit). The catalogue is accompanied by a `README.md` and a Python reader module `choreo_catalogue.py` for programmatic access.

9 Development Milestones

Milestone	Deliverable	Target
M0	Environment setup, JAX GPU verify, test on figure-8	Week 1
M1	C_{3v} and D_3 search complete ($N = 3$)	Week 2
M2	All $N = 3$ and $N = 4$ point groups searched	Week 4
M3	Knot pipeline validated; classification of M1/M2 results	Week 5
M4	Stability analysis complete for all discovered orbits	Week 6
M5	$N = 6, 12$ (tetrahedral, octahedral) search complete	Week 9
M6	Icosahedral $N = 12$ attempt	Week 11
M7	Visualisation pipeline, 4K renders, static figures	Week 12
M8	Catalogue finalised, toolkit released, paper drafted	Week 14

References

[1] H. Yoshida, *Construction of higher order symplectic integrators*, Phys. Lett. A **150** (1990), 262–268.

[2] A. Chenciner and R. Montgomery, *A remarkable periodic solution of the three-body problem in the case of equal masses*, Ann. Math. **152** (2000), 881–901.

[3] C. Simó, *New families of solutions in N-body problems*, in *European Congress of Mathematics*, Birkhäuser, Basel, 2001, pp. 101–115.

[4] R. S. Palais, *The principle of symmetric criticality*, Commun. Math. Phys. **69** (1979), 19–30.

[5] e5bed97b3349, *orbraid Mathematical Framework*, Technical Report v0.1, https://github.com/e5bed97b3349/orbraid, 2026.